

金融資料探勘-個人作業報告

組別:第 4 組

負責年度:2019

系級:財金三

學號:411236022

姓名:鄭廷宸

1. 作業目的

說明如何透過 Python 將 TEJ 下載的原始交易資料（加權指數 S0），依照台灣期交所的結算規則，自動化轉化為後續分析所需的選擇權索引檔。

2. 原始資料說明

- 資料來源：TEJ Pro 資料庫 — 上市/櫃指數資料。
- 選取標的：加權指數（Y9999）。
- 時間範圍：2019/01/01 至 2019/12/31。
- 原始欄位：日期、收盤價。

3. 資料建構流程



階段一：輸入與清洗

讀取原始 Excel 後，首先進行「數據去噪」。針對 TEJ 檔案特有的文字列（如 Y9999 標題）進行過濾，並將純數字或異常格式轉化為標準的 datetime 物件，避免出現 1970 年的預設日期。

階段二：金融邏輯判定

這是本作業的核心。透過演算法精確定位 2019 年每個月的第三個星期三。

關鍵規則：為了對齊範例圖片，我們實作了「結算日當天即換約」的邏輯。例如 1/16 為 1 月結算日，則 1/16 當天的資料就必須對應到 2 月合約，這會導致 \$Maturity\$ 在這一天發生劇烈跳轉。

階段三：產出索引檔

最後補入無風險利率（Rf）並計算日曆天數差，自動產出符合規範的 7 個欄位。

1. **日期整理**：將 TEJ 原始日期字串轉換為 Python datetime 格式。
2. **建立 File 欄位**：依照 OptionsDaily_YYYY_MM_DD.csv 格式自動生成每日對應檔名。
3. **合約判定邏輯**：
 - 定義結算日：每月第三個星期三。
 - 實作結算日換約規則：若「交易日 $\geq$ 當月結算日」，則合約（Contract）自動跳轉至下個月。
4. **計算 Maturity**：計算交易日至到期日之間的「日曆日」天數差。
5. **補入 Rf**：加入 2019 年無風險利率（參考值 0.0104）。
6. **補充**：針對 2019 年 2 月農曆春節之長假休市，索引檔依據實際交易日呈現，雖交易日期不連續，但 \$Maturity\$（到期天數）仍嚴格依據日曆日扣除，確保評價邏輯正確。

4. 最終索引檔欄位說明

欄位名稱	型態	用途
Date	Date (YYYY/MM/DD)	交易日期
File	String	每日選擇權明細檔之索引檔名

欄位名稱	型態	用途
S0	Float	標的加權指數收盤價
Maturity	Integer	距離到期日之日曆天數
Contract	String (YYYYMM)	當前交易之近月合約月份
ContractExpiry Date	Date (YYYY/MM/DD)	該合約之最後結算日
RiskFreeRate	Float	無風險利率 (Rf)

5. 成果展示（前 5 筆資料預覽）

Date	File	S0	Maturity	Contract	ContractExpiry Date	RiskFreeRate
2019/01/02	OptionsDaily_2019_01_02.csv	9554.14	14	201901	2019/01/16	0.0104
2019/01/03	OptionsDaily_2019_01_03.csv	9492.42	13	201901	2019/01/16	0.0104
2019/01/04	OptionsDaily_2019_01_04.csv	9382.51	12	201901	2019/01/16	0.0104
2019/01/07	OptionsDaily_2019_01_07.csv	9590.3	9	201901	2019/01/16	0.0104
2019/01/08	OptionsDaily_2019_01_08.csv	9563.6	8	201901	2019/01/16	0.0104

6. 問題與修正

- 問題 1：資料解析錯誤 (DateParseError)
 - 描述：TEJ 原始檔包含說明文字導致 Python 無法辨識日期格式。

```

*** DateParseError                                Traceback (most recent call last)
/tmp/ipykernel_911/1373881654.py in <cell line: 0>()
    17 df.columns.values[1] = '50'
    18
--> 19 df['Date'] = pd.to_datetime(df['Date'])
    20 df = df.sort_values('Date').reset_index(drop=True)
    21

~*~ 3 frames ~*~
/usr/local/lib/python3.12/dist-packages/pandas/core/arrays/datetimes.py in objects_to_datetime64(data, dayfirst, yearfirst, utc,
errors, allow_object, out_unit)
    2396 data = np.asarray(data, dtype=np.object_)
    2397
-> 2398 result, tz_parsed = tslib.array_to_datetime(
    2399     data,
    2400     errors=errors,

tslib.pyx in pandas._libs.tslib.array_to_datetime()
tslib.pyx in pandas._libs.tslib.array_to_datetime()
tslib.pyx in pandas._libs.tslib.array_to_datetime()
conversion.pyx in pandas._libs.tslibs.conversion.convert_str_to_tsoobject()
parsing.pyx in pandas._libs.tslibs.parsing.parse_datetime_string()
parsing.pyx in pandas._libs.tslibs.parsing.dateutil_parse()

DateParseError: Unknown datetime string format, unable to parse: Y9999 加權指數, at position 0

```

- 修正： 檢查最上面幾列。如果有任何寫著「Y9999 加權指數」或「資料來源：TEJ」的文字列，直接整列刪除，確保 Excel 的第一列就是欄位名稱（如：日期、收盤價），第二列開始就是數據。

• 問題 2：1970 年異常日期

- 描述：日期轉換邏輯未對齊原始格式，導致出現 Unix 起始日期。
- 修正：指定 `pd.to_datetime` 的格式為 `%Y/%m/%d` 以確保解析正確。

7. AI Prompt 實作與優化紀錄摘要

1. 任務目標

利用 AI 協助建構 2019 年份選擇權分析索引檔，核心要求包含：自動判定每月第三個週三為結算日、計算日曆日到期天數（Maturity），以及落實「結算日當天即自動換約」之金融實務邏輯。

2. 指令迭代過程

在實作過程中，我透過三次主要的優化來修正程式碼：

階段	指令重點 (Prompt)	AI 回應狀況與修正點
初始指令	要求寫出 2019 年索引檔，包含日期、收盤價、到期日。	產出的日期轉換邏輯太簡單，無法正確處理 TEJ 原始檔上方的文字說明，導致程式報錯。
優化指令	加入「自動跳過文字行」與「指定日期格式」要求。	解決了 DateParseError，但發現換約邏輯 (Roll-over) 錯誤，未能在結算日當天正確切換至下個月合約。
最終指令	強調「當日期 >= 當月結算日，合約與到期日需立即跳轉至次月」。	成功對齊範例圖片邏輯，產出符合規範的 7 個欄位，並解決 1970 年異常日期問題。

3. 實作成果摘要

- **成功處理檔案異常**：透過 Prompt 要求 AI 加入 skiprows 與 dropna 邏輯，排除 TEJ 檔案中的非數據列（如「Y9999 加權指數」文字）。
- **精準邏輯運算**：產出的 Maturity 嚴格遵循日曆日計算，並在 2019/01/16 結算日當天正確將 Contract 標記為 201902，到期天數亦同步由 1 天跳轉至 35 天。

4. 最佳指令實例 (The Master Prompt)

「請讀取『資料來源 1.xlsx』，第一欄為日期 (YYYY/MM/DD)，第二欄為收盤價。請自動計算 2019 年每月第三個週三為結算日。核心邏輯：若交易日大於或等於當月結算日，則 ContractExpiry Date 須設為下個月結算日。輸出欄位須包含：Date, File, S0, Maturity, Contract, ContractExpiry Date, RiskFreeRate (0.0104)。」

8. 學習反思

1. AI 提供的具體幫助

這是我第一次嘗試將 Python 應用在處理這麼大量的金融資料上。對於大三生來說，手動計算 2019 年整年 240 多天的結算日和 Maturity（到期天數）幾乎是不可能的任務，不僅耗時且極易出錯。

透過與 AI 的對話，我成功寫出了自動化腳本。AI 幫我解決了幾個最棘手的邏輯問題：

- **自動判斷結算日**：精準計算出每個月的「第三個星期三」，不需要對著農曆表慢慢查。
- **實作換約規則**：AI 幫我寫出了「當日 $\geq$ 結算日就跳合約」的邏輯，讓資料能在結算當天平滑地移轉到下個月，這與範例圖片的標準完全一致。

2. 遇到的困難與 AI 的限制

雖然 AI 很強大，但在實作過程中我發現它並不是「全能」的，它需要非常具體的指令：

- **環境差異問題**：一開始我直接跑程式卻出現 `FileNotFoundError`，後來才發現是因為 Google Colab 是雲端環境，必須手動上傳檔案。
- **原始資料格式不一**：從 TEJ 下載的原始資料上方帶有說明文字，導致 AI 最初寫的程式碼出現 `DateParseError`。AI 雖然知道怎麼處理日期，但它並不知道我的 Excel 檔案結構長什麼樣子，這讓我意識到**「資料清洗 (Data Cleaning)」**才是程式運行成功的前提。

3. Prompt 修正的心得與反思

這次作業讓我學到最重要的不是程式碼本身，而是如何精確地與 AI 溝通 (Prompt Engineering)。

- **從模糊到具體**：一開始我的 Prompt 很簡單，AI 產出的結果日期格式亂掉，且出現了 1970 年的異常數據。
- **加入限制條件**：當我加入「指定日期格式 YYYY/MM/DD」以及「強制定義

第一欄為日期、第二欄為收盤價」後，程式碼的穩定度大幅提升。

4. 未來改進方向

目前雖然能成功產出索引檔，但如果未來要處理三年的整合資料，我會希望學習如何讓 AI 幫我寫出更具「模組化」的程式碼，自動偵測各年度的農曆春節休市日，並加入更嚴謹的缺值檢查 (NaN Check)，確保合併後的資料品質一致。